

Assistant AI – AI- powered FAQ and customer support automation

Description

Makes the system flexible and easier to maintain. The AI FAQ Assistant API is an intelligent and automated question-answering system developed to provide users with quick, accurate, and relevant answers to frequently asked questions. The system uses Artificial Intelligence (AI) and Natural Language Processing (NLP) techniques to understand the user's questions and identify the appropriate response from the available FAQ information.

The API acts as a communication layer between the user interface and the AI-based FAQ system. When a user enters a question, the request is sent to the API. The system processes the question, identifies the important keywords and meaning, and searches the available FAQ data for the most relevant answer. The response is then returned to the application and displayed to the user in an understandable format.

The AI FAQ Assistant API can be integrated with websites, mobile applications, educational platforms, business applications, and customer-support systems. It can answer common questions related to products, services, courses, policies, registration, payments, technical support, and other frequently requested information.

One of the major advantages of this system is that it provides 24/7 automated support without requiring a human representative for every basic query. It can reduce the workload of customer-support staff and improve the overall user experience by providing responses quickly.

The system can also be designed to allow administrators to add, update, and remove FAQ questions and answers. This As the FAQ database grows, the assistant can handle a larger number of questions and provide users with more useful information.

Scenario- Based case study

Background

To overcome these challenges, **Artificial Intelligence (AI)** can be used to automate the process of answering frequently asked questions. An **AI FAQ Assistant API** provides a way for applications to communicate with an intelligent FAQ system. It receives questions from users, processes the input, understands the meaning of the query using **Natural Language Processing (NLP)**, and provides a relevant answer.

Problem

- **🔍 Automated Question Answering:** Provides instant answers to frequently asked questions.
- **🧠 AI-Based System:** Uses Artificial Intelligence to understand and process user queries.
- **🗣️ Natural Language Processing:** Understands questions even when users phrase them in different ways.
- **🔗 API Integration:** Can be integrated with websites, mobile applications, and other software systems.
- **📝 FAQ Management:** Allows administrators to add, update, and delete questions and answers.

Solution

The proposed solution is to develop an AI-powered FAQ Assistant API that automatically understands user questions and provides relevant answers. The system uses Artificial Intelligence and Natural Language Processing (NLP) to process user queries instead of depending only on exact keyword matching.

The proposed solution is to develop an **AI-powered FAQ Assistant API** that can automatically understand and respond to users' frequently asked questions. The system uses **Artificial Intelligence (AI) and Natural Language Processing (NLP)** to analyse user queries and identify the most relevant answers from a predefined FAQ knowledge base. Instead of requiring users to manually search through large amounts of information, the system provides quick, understandable responses.

Usage

- Admin: Manages users, FAQ, categories, and monitors AI-generated content.
- Content Creator: Creates, edits, deletes, organises, and generates FAQs using AI.
- Authenticated user: views FAQs, searches for information, and generates AI-assisted answers.
- Public user: Views and searches published FAQs without protected operations.

Outcome

The outcome of the AI FAQ Assistant API is to provide an intelligent, fast, and user-friendly system for answering frequently asked questions automatically. The system successfully processes user queries using Artificial Intelligence and Natural Language Processing and provides relevant answers from the available FAQ information. It reduces the need for manual customer support and allows users to obtain information quickly without searching through large amounts of content. The system also provides 24/7 assistance, improves response time, reduces repetitive work, and enhances the overall user experience. Administrators can easily maintain and update the FAQ database whenever new information is available. Overall, the project demonstrates how AI and API technologies can be combined to create an efficient and scalable automated support system.

1. Software Requirements

- **Operating System** – Windows 10/11, Linux, or macOS.
- **Programming Language** – Python 3.x for developing the API and backend logic.
- **Framework** – Flask or Django for building and managing the API.
- **Database** – MySQL or SQLite for storing FAQs, questions, answers, and user-related information.
- **API Technology** – REST API for communication between the FAQ system and client applications.
- **AI/NLP Libraries** – Natural Language Processing libraries such as NLTK, spaCy, or suitable AI/ML APIs.
- **Code Editor** – Visual Studio Code, PyCharm, or any Python-supported IDE.
- **Web Browser** – Google Chrome, Microsoft Edge, or Mozilla Firefox for testing the application.

2. Hardware Requirements

- **Processor:** Intel Core i3 or above is required for smooth application development.
- **RAM:** A minimum of 4 GB RAM is required to run the software. 8 GB RAM is recommended for better performance.
- **Storage:** At least 10 GB of free storage space is required. Additional space may be needed for project files and databases.
- **Monitor:** A standard HD monitor with 1366 × 768 resolution or above is recommended.
- **Internet Connection:** A stable internet connection is required for API integration. It is also useful for downloading required packages and services.

Request Flow

The request flow of the **AI FAQ Assistant API** begins when the user enters a question through the application. The question is sent to the API, where it is processed using Natural Language Processing techniques to understand the user's query. The system then searches the FAQ database or knowledge base to find the most relevant answer. After identifying the appropriate response, the API sends the answer back to the application. Finally, the user can view the generated response quickly and easily. This process provides an efficient and automated way to handle frequently asked questions. The request flow of the **AI FAQ**

Assistant API describes how a user's question is received, processed, and answered by the system. The process begins when the user enters a question or selects a frequently asked question through the application interface. The request is then sent to the AI FAQ Assistant API using a suitable HTTP request.

Architecture Flow

API Client → Express Server → Authentication / Application Services → Database Layer

Application Services → Google Gemini AI → Generated FAQ Content

This layered structure keeps the system modular and makes the backend easier to maintain, extend, secure, and test.

Entity Relationship Diagram Description

The **Entity Relationship Diagram (ERD)** represents the relationship between the different entities used in the **AI FAQ Assistant API**. The main entity is the **FAQ**, which stores frequently asked questions and their corresponding answers. Each FAQ can contain details such as FAQ ID, question, answer, category, and creation date.

User Entity

Stores user details such as User ID, name, email, and login information.

Main attributes:

1. FAQ Entity

The FAQ entity stores the questions and answers managed by the application.

Main attributes:

- FAQ ID
- Question
- Answer
- Category

- Created By
- Created Date
- Updated Date

Each FAQ is associated with a user who creates it and is organized under a category.

2. Category Entity

The Category entity is used to group FAQs according to their topics or subject areas.

Main attributes:

- Category Name
- Description
- Total FAQs

One category can contain multiple FAQ records.

3. AI Generation Entity

AI Generation represents the logical data produced when the system uses Google Gemini to generate FAQ content. This information can later be stored as an FAQ record.

Main attributes:

- Topic
- Generated Question
- Generated Answer
- Generated Category
- Generated Date

A user can generate multiple AI-assisted FAQ entries.

Relationship Description

User → FAQ (One-to-Many):

A single user can create and manage multiple FAQ records, while each FAQ is associated with its creator.

Category → FAQ (One-to-Many):

A category can contain several FAQ records, allowing related questions and answers to be organized together.

User → AI Generated FAQ (One-to-Many):

A user may generate multiple FAQ contents using the AI service. These generated results can later be saved as FAQ records.

FAQ → Search Results (Logical Relationship):

FAQ records can be retrieved through keyword or semantic search based on the user's query.

Overall, the ER diagram provides a structured representation of the data relationships that support authentication, FAQ management, category organization, AI-assisted generation, and information retrieval.

Key Structural Mappings

The database entities of the AI FAQ Assistant are connected through logical relationships that support user management, FAQ organization, and AI-assisted content generation.

User – FAQ Relationship

A registered user can be associated with multiple FAQ records. Each FAQ maintains a reference to the user responsible for creating or managing it.

Category – FAQ Relationship

A single category can group several FAQ entries. This relationship helps organize questions and answers according to their respective topics.

User – AI Generated FAQ Relationship

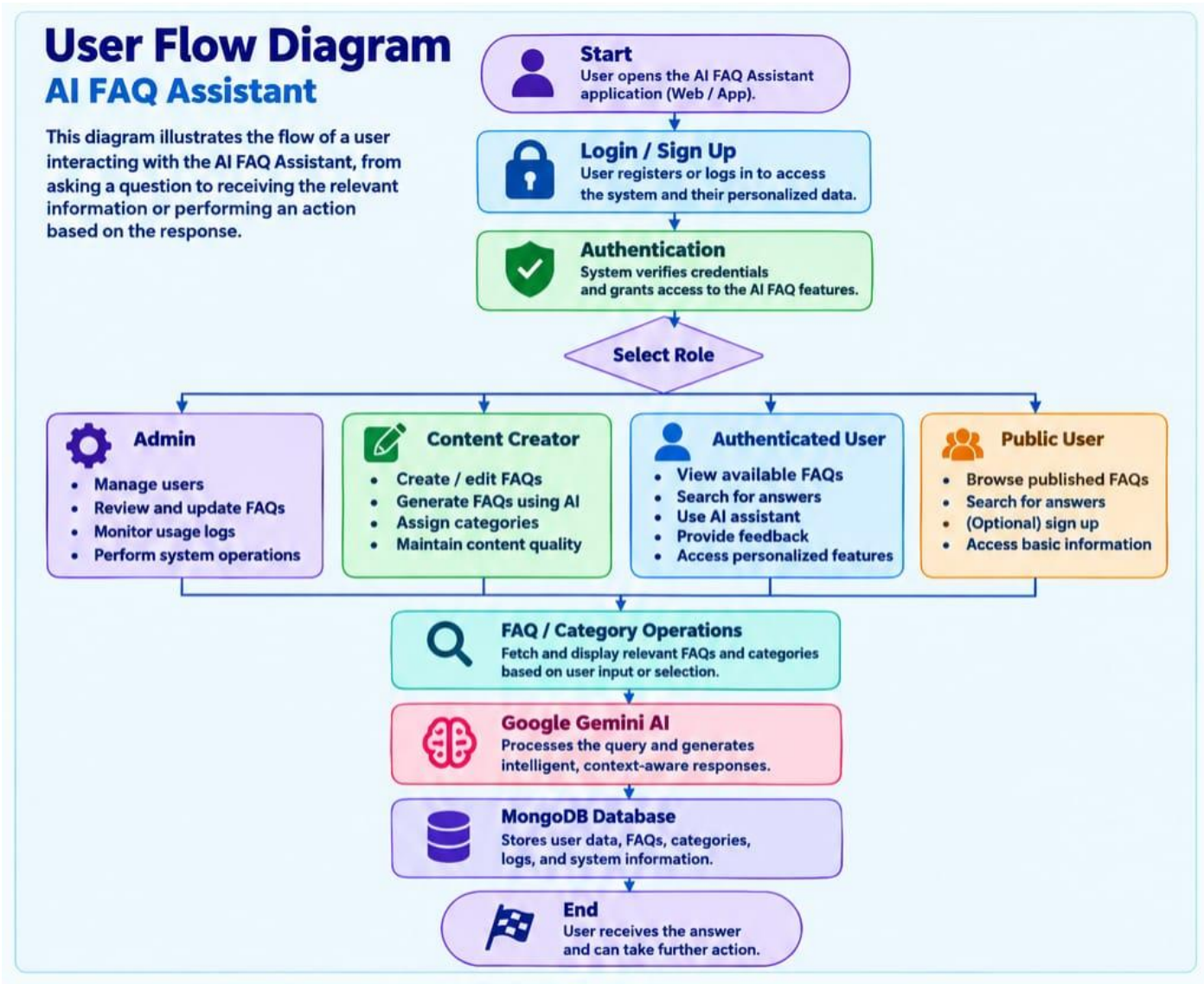
One user may generate multiple FAQ contents with the assistance of the AI service. The generated information can subsequently be stored as FAQ data.

FAQ – Search Relationship

FAQ records are accessed through search operations. User queries can retrieve relevant FAQ information using keyword-based or semantic matching.

These mappings provide the logical connections required for maintaining organized FAQ

data, associating content with users and categories, and supporting AI-assisted FAQ.



Architecture Pattern

The AI FAQ Assistant is designed using the Model–View–Controller (MVC) architecture pattern. This architecture separates the application into different layers, making the system easier to develop, maintain, test, and manage. Each layer performs a specific function and communicates with the other layers through defined processes.

1. Model

The Model layer is responsible for handling application data and database-related activities. It uses Mongoose to create data schemas and communicate with the MongoDB database.

Responsibilities:

Store User, FAQ, and Category information

Define database schemas and structures

Perform database operations

Validate and manage stored information

Retrieve, update, and maintain application records

2. Controller

The Controller layer acts as a connection between the API requests and the application's main business logic. It receives requests from users and processes them according to the required operation.

Responsibilities:

Receive and handle API requests

Check and validate input data

Connect with services and database operations

Process the requested information

Send suitable responses to the user

3. View

Since the AI FAQ Assistant is developed as a REST API, it does not use a traditional graphical View layer. Instead, the View is represented through API routes and JSON responses returned to the client.

Responsibilities:

Provide API endpoints

Return information in JSON format

Display success and error responses

Send processed results to API clients such as Postman or other applications

4. Supporting Components

The MVC architecture is further supported by several backend components that improve the functionality and security of the system.

Authentication Middleware – Verifies JWT tokens and controls access to protected resources.

Service/Business Logic Layer – Handles FAQ management, searching, category operations, and AI-based FAQ generation.

AI Integration – Connects the application with Google Gemini to generate FAQ content and intelligent responses.

Error Handling – Manages errors in a centralized manner and provides appropriate API error responses.

Architecture Flow

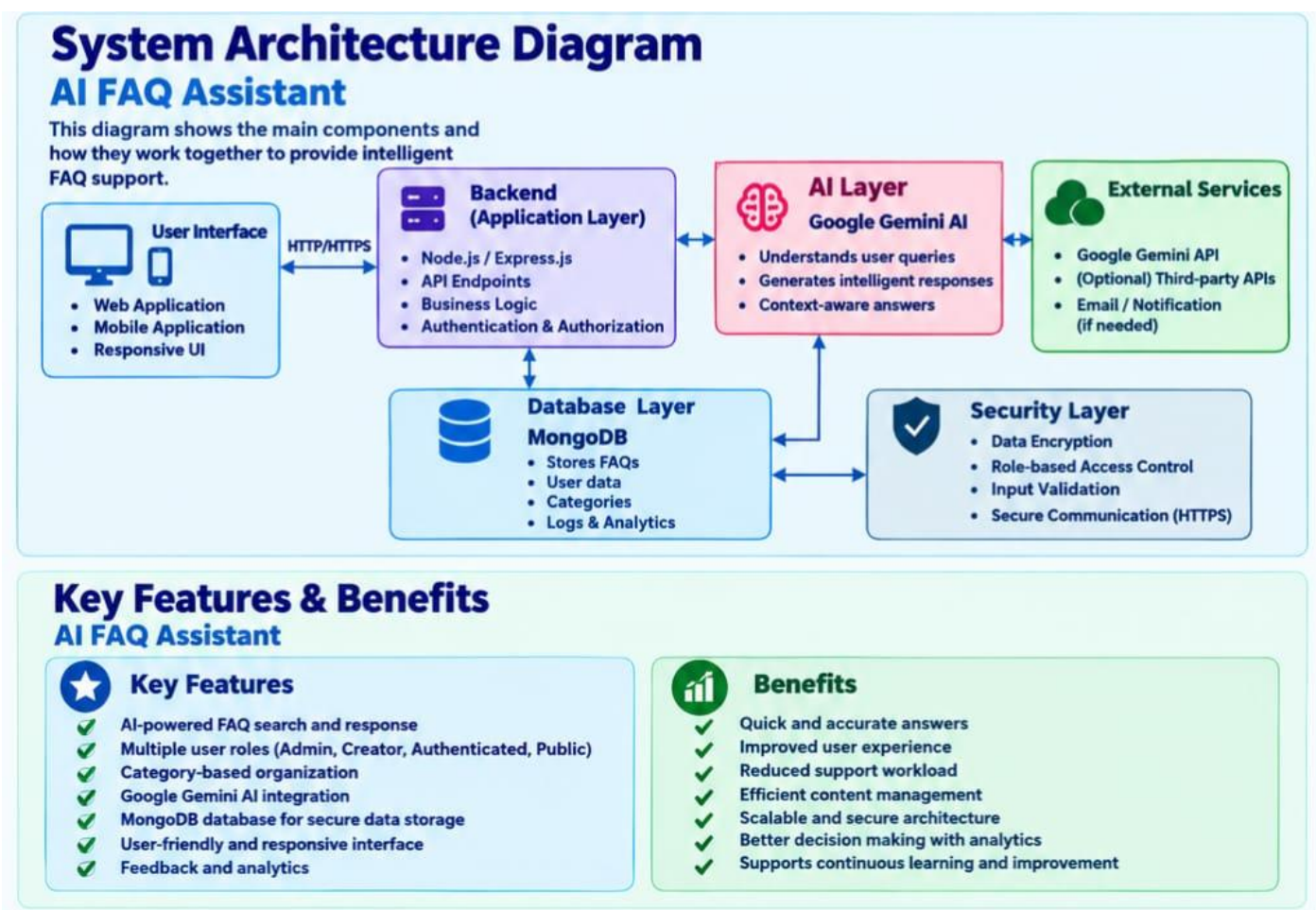
The main request flow of the application can be represented as:

API Client → Routes → Middleware → Controller → Service/Logic → Model → MongoDB

For AI-based FAQ generation, the process is:

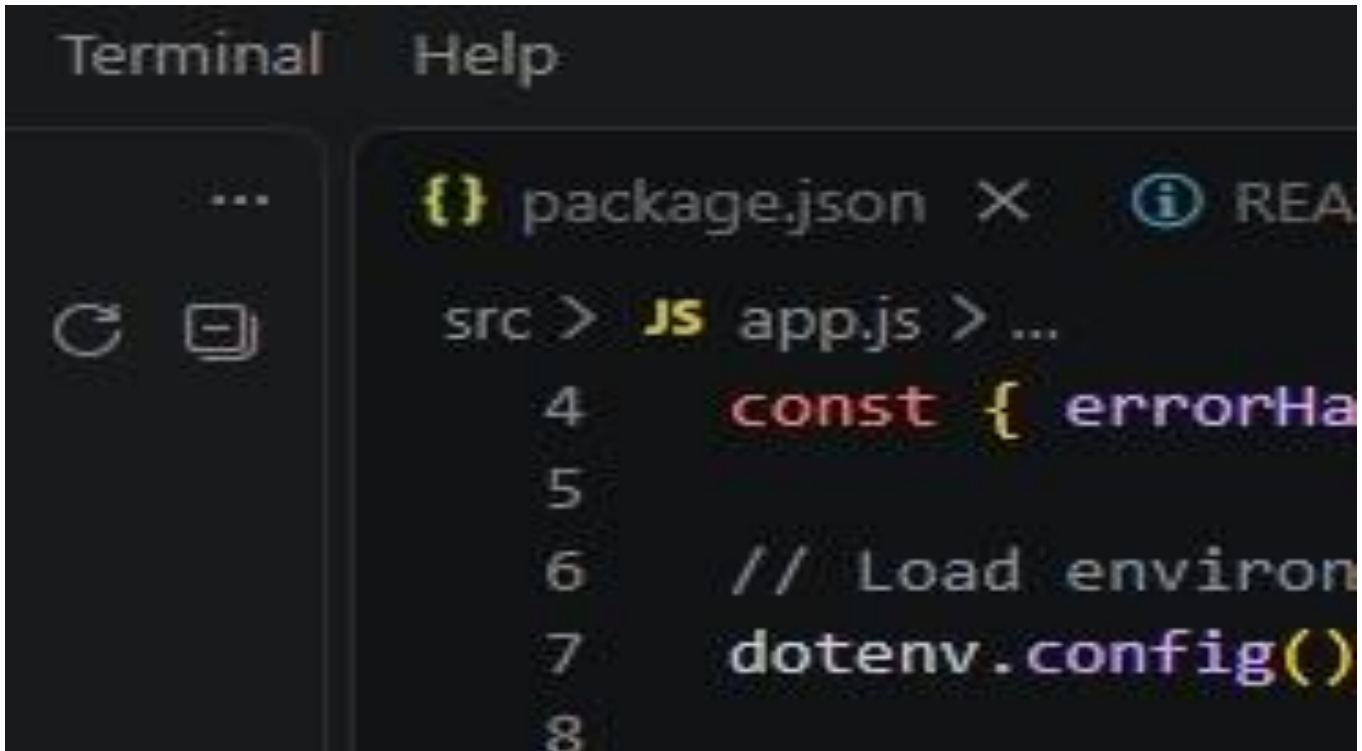
API Client → Route → Controller → AI Service → Google Gemini → Response

This architecture provides a clear separation of responsibilities and makes the AI FAQ Assistant API easier to maintain, extend, and integrate with other applications.



Epic 2: Project Setup and Configuration

The AI FAQ Assistant API is developed using a structured backend environment. This phase includes setting up the project workspace, installing the required packages, organizing source files, and preparing the application for further development.



Project Initialization

The project is created as a separate backend application to maintain a clear and organized development structure.

Main activities:

- Create the AI FAQ Assistant project directory

- Configure the Node.js environment

- Install required backend packages

- Create the src folder for source files

- Configure environment variables

- Prepare the application for database connectivity

Epic 3: Backend Development

The backend is the main processing part of the AI FAQ Assistant. It manages API requests, authentication, FAQ operations, database communication, and AI-based services.

1. Server Configuration

The server file is responsible for starting the backend application and preparing the required services.

Key functions:

Load application settings

Establish database connectivity

Initialize the Express server

Start the application on the configured port

2. API Application Setup

The Express application manages incoming requests and provides the required API structure.

Key functions:

Configure Express

Enable JSON data processing

Register API routes

Connect middleware components

Handle client requests

3. Authentication Module

The authentication module provides security for protected API resources. It checks the user's authentication token before allowing access.

Key functions:

Receive authentication token

Validate JWT credentials

Identify the logged-in user

Allow authorized requests

Block unauthorized access

4. Error Management

The error management component handles problems that occur during API execution and database operations.

Key functions:

Detect application errors

Handle database-related failures

Generate proper error responses

Maintain consistent API responses

Avoid exposing unnecessary system information

Backend Processing Flow

Client → Express Server → Authentication → API Route → Controller → Service → Database / AI Service → Response

Epic 4: Database Configuration

Database configuration

The AI FAQ Assistant uses MongoDB for storing application information and Mongoose for managing communication between the backend and database.

1. Database Connectivity

The application establishes a connection with MongoDB through Mongoose. Database configuration details are maintained separately from the main application code.

2. User Data Structure

The User collection maintains information related to registered users.

Important fields:

User ID

Name

Email

Password

User Role

Created Date

Updated Date

Passwords are securely protected before being stored in the database.

3. FAQ Data Structure

The FAQ collection stores the questions and answers used by the application.

Important fields:

FAQ ID

Question

Answer

Category

Created By

Created Date

Updated Date

The Created By field identifies the user responsible for creating the FAQ.

4. Data Verification

Mongoose validation rules are applied to ensure that required information is available before saving records. This helps maintain accurate and consistent database data.

5. Data Relationship

The main relationship between the collections is:

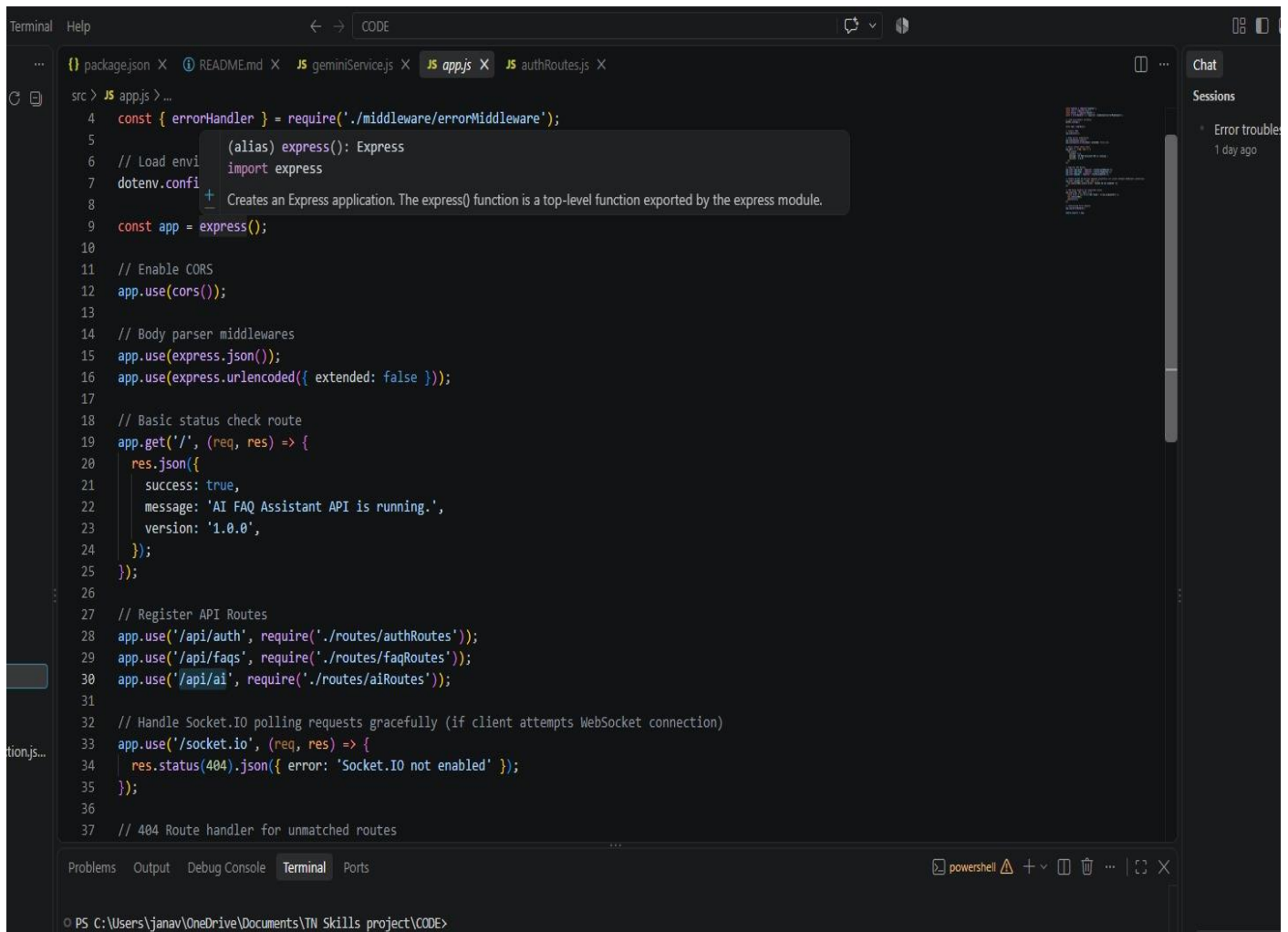
User → FAQ (One-to-Many)

A single user can create and manage multiple FAQ records.

Database Processing

Application → Mongoose → MongoDB

This setup allows the system to store, access, modify, and manage user and FAQ information efficiently.



```
src > JS app.js > ...
4  const { errorHandler } = require('./middleware/errorMiddleware');
5
6  // Load environment variables
7  dotenv.config();
8
9  const app = express();
10
11 // Enable CORS
12 app.use(cors());
13
14 // Body parser middlewares
15 app.use(express.json());
16 app.use(express.urlencoded({ extended: false }));
17
18 // Basic status check route
19 app.get('/', (req, res) => {
20   res.json({
21     success: true,
22     message: 'AI FAQ Assistant API is running.',
23     version: '1.0.0',
24   });
25 });
26
27 // Register API Routes
28 app.use('/api/auth', require('./routes/authRoutes'));
29 app.use('/api/faqs', require('./routes/faqRoutes'));
30 app.use('/api/ai', require('./routes/aiRoutes'));
31
32 // Handle Socket.IO polling requests gracefully (if client attempts WebSocket connection)
33 app.use('/socket.io', (req, res) => {
34   res.status(404).json({ error: 'Socket.IO not enabled' });
35 });
36
37 // 404 Route handler for unmatched routes
```

API Router and Endpoint Implementation

The AI FAQ Assistant API uses a modular routing structure to manage authentication, FAQ operations, and AI-based FAQ generation. Each route is responsible for a specific functionality of the application. This structure makes the backend organized, easy to test, and simple to maintain.

1. Authentication API

File: src/routes/authRoutes.js

The authentication router manages user account creation and login activities. These endpoints are available without authentication because they are required for creating an account and obtaining access credentials.

Main Endpoints

POST /api/auth/register – Creates a new user account.

POST /api/auth/login – Verifies the user's login details and generates a JWT authentication token.

The generated token is used to access protected resources in the application.

2. FAQ Management API

File: src/routes/faqRoutes.js

The FAQ router handles the creation and management of FAQ records. Operations that modify FAQ information require a valid Bearer JWT token.

Main Operations

Create new FAQ records

Retrieve available FAQs

Update existing FAQ information

Delete unwanted FAQ records

Search FAQs using keywords or query parameters

Example Endpoint

POST /api/faqs

Access: Private – Bearer JWT Token

This endpoint accepts FAQ details from an authenticated user and stores the information in the MongoDB database.

3. AI-Based FAQ Generation

File: src/routes/aiRoutes.js

The AI router provides automatic FAQ generation using the integrated Google Gemini SDK. It allows authenticated users to submit a topic and receive automatically generated FAQ content.

Main Endpoint

POST /api/ai/generate-faq

Access: Private – Bearer JWT Token

The request must contain a valid authentication token. After authentication, the submitted topic is passed to the AI service, which communicates with Google Gemini to generate relevant FAQ content.

Request Data

The endpoint accepts a topic or context in the request body.

Example:

```
{  
  "topic": "MongoDB database management"  
}
```

AI Processing Flow

Client Request → AI Route → Authentication Check → AI Service → Google Gemini → Generated FAQ Response

The generated FAQ information is then returned to the client as an API response.

4. API Execution and Testing

The developed API can be tested using tools such as Postman or Thunder Client. These tools allow different HTTP requests to be sent to the configured endpoints and help verify the returned responses.

Starting the Backend

Open the project terminal and execute:

```
npm run dev
```

This command starts the development server and makes the API endpoints available for testing.

Testing Sequence

Start Server → User Registration → User Login → Receive JWT Token → FAQ Operations → Search FAQ → AI FAQ Generation → JSON Response

5. Overall API Flow

The complete backend request process can be represented as:

Client → Express Router → Authentication Middleware → Controller → Service/Logic → MongoDB / Google Gemini → Response

For normal FAQ operations, the request is processed through the database layer. For AI-based operations, the request is forwarded to the AI service and then to Google Gemini.

Purpose

This routing design separates user authentication, FAQ management, database operations, and AI processing into individual modules. As a result, the AI FAQ Assistant API has a clear backend structure and can be easily extended with additional features in the future.

Epic 5: API Execution and Testing

The AI FAQ Assistant API is tested to verify that authentication, FAQ management, search operations, and AI-based FAQ generation work correctly. API requests can be sent through tools such as Postman or Thunder Client.

1. API Testing Environment

Postman or Thunder Client can be used to send requests to the backend API and check the returned responses.

2. Start the Application

Open the project terminal and run:

This starts the Node.js backend and makes the configured API services available for testing.

API Operations

1. Register User

Method: POST

Endpoint: /api/auth/register

Access: Public

This operation is used to create a new account in the system.

Sample Data:

2. Authenticate User

Method: POST

Endpoint: /api/auth/login

Access: Public

The login operation checks the registered user's credentials and returns an authentication token when the details are valid.

Sample Data:

3. Add FAQ

Method: POST

Endpoint: /api/faqs

Access: Authenticated User

This operation allows an authorized user to add a new question and answer to the FAQ database.

Sample Data:

4. Find FAQ

Method: GET

Endpoint: /api/faqs/search?q=account

Access: Public

This operation searches the FAQ collection using a keyword and returns the matching FAQ information.

5. Modify FAQ

Method: PUT

Endpoint: /api/faqs/:id

Access: Authenticated User

This operation allows an authorized user to modify an existing FAQ record.

6. Remove FAQ

Method: DELETE

Endpoint: /api/faqs/:id

Access: Authenticated User

This operation deletes a selected FAQ record from the database.

AI FAQ Generation

The application provides an AI-based FAQ generation feature using **Google Gemini**. An authenticated user can submit a topic, and the AI service generates suitable FAQ content.

Method: POST

Endpoint: /api/ai/generate-faq

Access: Private

Input Data

The request contains a topic or subject for which the FAQ should be generated.

Example:

AI Processing Flow

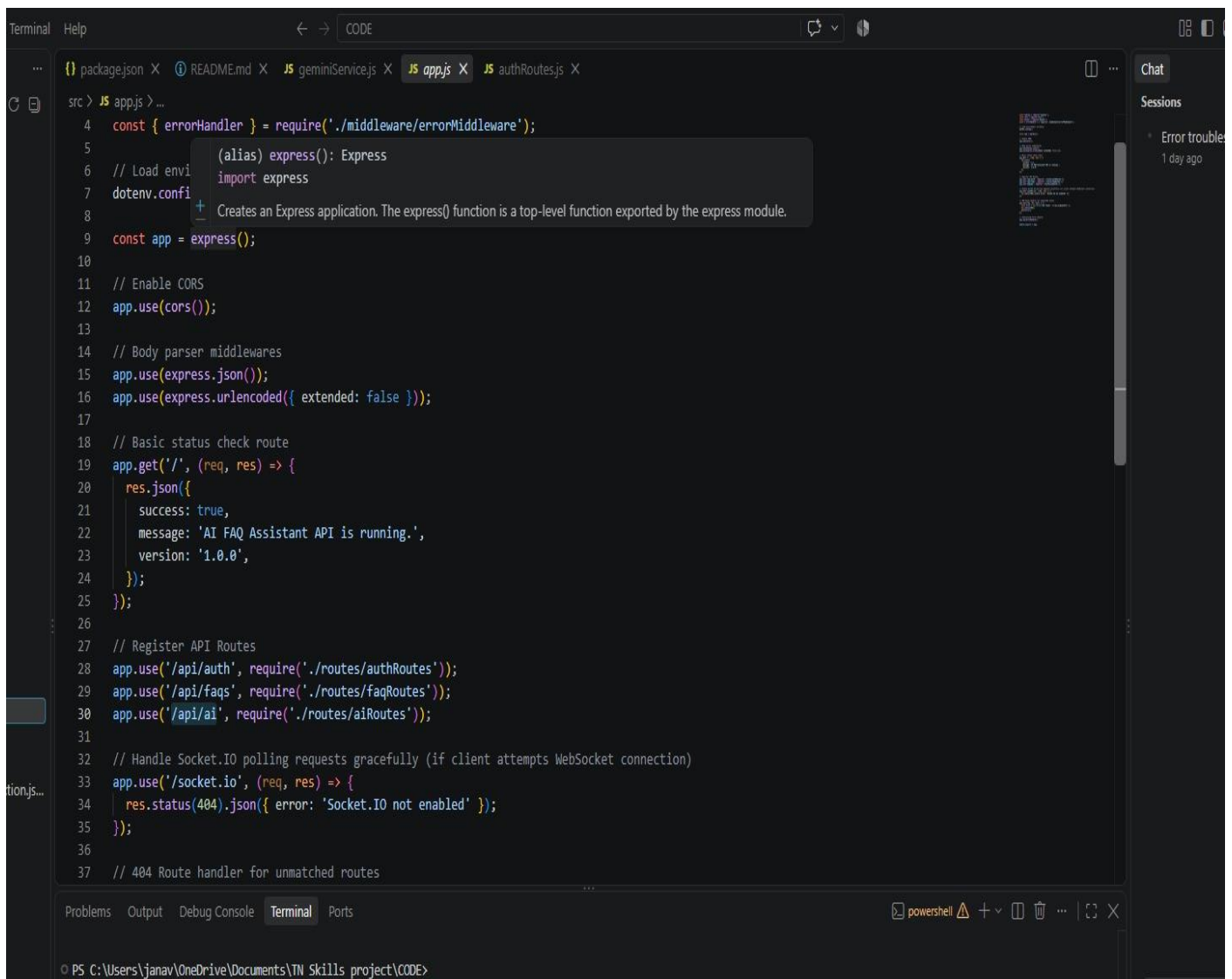
**User Request → API Route → Authentication → AI Controller → Gemini Service →
Generated FAQ → JSON Response**

The generated information is returned to the client in JSON format.

Overall Execution Flow

**Start Application → User Authentication → FAQ Operation → Database Processing → AI
Generation if Required → API Response**

This testing and execution process verifies that the major modules of the **AI FAQ Assistant API** communicate correctly and perform their assigned functions.



The screenshot shows a VS Code editor with the following code in `app.js`:

```
1  const { errorHandler } = require('./middleware/errorMiddleware');
2
3  // Load environment variables from .env
4  import express
5  // Create an Express application. The express() function is a top-level function exported by the express module.
6  const app = express();
7
8  // Enable CORS
9  app.use(cors());
10
11 // Body parser middlewares
12 app.use(express.json());
13 app.use(express.urlencoded({ extended: false }));
14
15 // Basic status check route
16 app.get('/', (req, res) => {
17   res.json({
18     success: true,
19     message: 'AI FAQ Assistant API is running.',
20     version: '1.0.0',
21   });
22 });
23
24 // Register API Routes
25 app.use('/api/auth', require('./routes/authRoutes'));
26 app.use('/api/faqs', require('./routes/faqRoutes'));
27 app.use('/api/ai', require('./routes/aiRoutes'));
28
29 // Handle Socket.IO polling requests gracefully (if client attempts WebSocket connection)
30 app.use('/socket.io', (req, res) => {
31   res.status(404).json({ error: 'Socket.IO not enabled' });
32 });
33
34 // 404 Route handler for unmatched routes
```

A tooltip for the `express()` function is visible, stating: "Creates an Express application. The express() function is a top-level function exported by the express module."

3. FAQ Record Management

Endpoint: POST `/api/faqs`

Authorization: Bearer JWT Token

This API is responsible for adding a new FAQ entry to the application. The service is available only to users who have successfully authenticated with a valid JWT token.

The user provides the required question, answer, and category details. The submitted information is checked before it is passed to the Mongoose model and stored in MongoDB.

Processing Flow:

User Request → JWT Verification → Input Checking → FAQ Controller → Mongoose Model → MongoDB → API Response

This process ensures that FAQ records are created only by authorized users and that the stored information follows the required database structure.

4. FAQ Keyword Search

Endpoint: GET /api/faqs/search?q=category

Authorisation: Public

This API allows users to find FAQ information by entering a relevant keyword. Authentication is not required for this search operation.

The server receives the keyword from the query parameter and compares it with the available FAQ records. Matching results are then returned to the user in JSON format.

Sample Request:

GET /api/faqs/search?q=category

Processing Flow:

Search Request → Query Parameter → Search Controller → Database Query → Matching FAQs → JSON Result

This feature makes FAQ retrieval easier by allowing users to quickly locate information related to a particular keyword.

